

Занятие 2: Маршрутизация и обработка запросов

Routing, HTTP-методы, обработчики запросов

Слайд 1: Что такое маршрутизация (Routing)?

Маршрутизация - процесс определения того, какой обработчик должен выполнить запрос на основе URL и HTTP-метода.

Компоненты маршрутизации:

- **URL Pattern** - шаблон URL
- **HTTP Method** - метод запроса (GET, POST, etc.)
- **Handler/View** - функция или класс, обрабатывающий запрос
- **Parameters** - параметры из URL или тела запроса

URL + HTTP Method → Router → Handler → Response

Слайд 2: HTTP-методы в backend

Метод	Описание	Idempotent	Использование
GET	Получение данных	Да	Чтение ресурсов
POST	Создание ресурса	Нет	Создание новых данных
PUT	Полное обновление	Да	Замена ресурса
PATCH	Частичное обновление	Нет	Изменение части ресурса
DELETE	Удаление	Да	Удаление ресурса
HEAD	Заголовки ответа	Да	Проверка существования
OPTIONS	Доступные методы	Да	CORS preflight

Слайд 3: Структура URL

`https://example.com/api/users/123/posts?sort=date&limit=10`

The diagram illustrates the structure of the URL `https://example.com/api/users/123/posts?sort=date&limit=10` using brackets and labels:

- `https://example.com` is labeled "Protocol + Domain".
- `/api` is labeled "Path".
- `/users/123` is labeled "Path Parameter".
- `/posts` is labeled "Resource".
- `?sort=date&limit=10` is labeled "Query Parameters".

Path Parameters: `/users/{id}` - часть URL

Query Parameters: `?page=1&limit=10` - после ?

Слайд 4: Параметры маршрута

Path Parameters (Django):

```
path('users/<int:user_id>/', views.user_detail)
# /users/123/ → user_id = 123
```

Path Parameters (FastAPI):

```
@app.get("/users/{user_id}")
def read_user(user_id: int):
    return {"user_id": user_id}
# /users/123 → user_id = 123
```

Query Parameters:

```
# /items?skip=0&limit=10
def read_items(skip: int = 0, limit: int = 10):
    return {"skip": skip, "limit": limit}
```

Слайд 5: Обработка тела запроса (Request Body)

JSON Body - стандарт для REST API:

```
{  
  "name": "John Doe",  
  "email": "john@example.com",  
  "age": 30  
}
```

Django:

```
data = json.loads(request.body)  
name = data.get('name')
```

FastAPI (с Pydantic):

```
from pydantic import BaseModel  
  
class User(BaseModel):  
    name: str  
    email: str  
    age: int
```

Слайд 6: Валидация данных

Зачем нужна валидация:

- Защита от некорректных данных
- Безопасность (SQL injection, XSS)
- Улучшение UX (понятные ошибки)

Типы валидации:

- Тип данных (int, str, email)
- Обязательные поля
- Диапазоны значений
- Формат данных (regex)

FastAPI с Pydantic:

```
class User(BaseModel):  
    name: str = Field(..., min_length=2, max_length=50)  
    email: EmailStr  
    age: int = Field(..., ge=18, le=120)
```

Слайд 7: Маршрутизация в Django

urls.py:

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.index, name='index'),
    path('users/', views.users_list, name='users_list'),
    path('users/<int:user_id>/', views.user_detail, name='user_detail'),
]
```

Включение app URLs в главный urls.py:

```
from django.urls import path, include

urlpatterns = [
    path('api/', include('myapp.urls')),
]
```

Слайд 8: Маршрутизация в FastAPI

Простые маршруты:

```
@app.get("/")
def root():
    return {"message": "Root"}

@app.get("/items/{item_id}")
def read_item(item_id: int):
    return {"item_id": item_id}
```

APIRouter для модульности:

```
from fastapi import APIRouter

router = APIRouter(prefix="/users", tags=["users"])

@router.get("/")
def get_users():
    return {"users": []}

@router.get("/{user_id}")
def get_user(user_id: int):
    return {"user_id": user_id}
```

Слайд 9: Обработка разных HTTP-методов

Django:

```
from django.views.decorators.http import require_http_methods

@require_http_methods(["GET", "POST"])
def user_list(request):
    if request.method == "GET":
        # Вернуть список
    elif request.method == "POST":
        # Создать пользователя
```

FastAPI:

```
@app.get("/users")
def get_users():
    return {"users": []}

@app.post("/users")
def create_user(user: UserCreate):
    return user

@app.put("/users/{user_id}")
def update_user(user_id: int, user: UserUpdate):
```

Слайд 10: Статус-коды ответов

Код	Класс	Значение	Использование
200	OK	Успех	GET, PUT, PATCH
201	Created	Ресурс создан	POST
204	No Content	Успех без тела	DELETE
400	Bad Request	Ошибка клиента	Неверные данные
404	Not Found	Не найдено	Ресурс не существует
405	Method Not Allowed	Метод не разрешен	Неверный HTTP-метод
500	Server Error	Ошибка сервера	Внутренняя ошибка

Слайд 11: Возврат статус-кодов

Django:

```
from django.http import JsonResponse, HttpResponseNotFound

def user_detail(request, user_id):
    user = get_user(user_id)
    if not user:
        return HttpResponseNotFound("User not found")
    return JsonResponse(user.to_dict())
```

FastAPI:

```
from fastapi import HTTPException

@app.get("/users/{user_id}")
def get_user(user_id: int):
    user = get_user(user_id)
    if not user:
        raise HTTPException(status_code=404, detail="User not found")
    return user
```

Слайд 12: Практическое задание

Задание 1: CRUD для пользователей

- GET /users/ - список всех пользователей
- GET /users/{id}/ - детальная информация
- POST /users/ - создание пользователя
- PUT /users/{id}/ - обновление
- DELETE /users/{id}/ - удаление

Задание 2: Фильтрация и сортировка

- GET /users/?age_gt=18&sort=name
- GET /users/?search=John

Задание 3: Валидация

- Имя: 2-50 символов
- Email: валидный email
- Возраст: 18-120

Слайд 13: Домашнее задание

1. Реализовать полноценный CRUD API для сущности "Product"
2. Добавить валидацию полей (название, цена, количество)
3. Реализовать фильтрацию по цене и категории
4. Добавить пагинацию (page, page_size)
5. Протестировать все endpoints через Postman
6. Создать README с описанием API

Слайд 14: Полезные ресурсы

Документация:

- Django URL Dispatcher: <https://docs.djangoproject.com/en/4.2/topics/http/urls/>
- FastAPI Path Parameters: <https://fastapi.tiangolo.com/tutorial/path-params/>
- HTTP Methods: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods>

Инструменты:

- Postman: <https://www.postman.com/>
- Insomnia: <https://insomnia.rest/>
- HTTPie: <https://httpie.io/>